

NLgov Assurance profile for OAuth 2.0



Logius Standard

Draft September 17, 2026

This version:

<https://logius-standaarden.github.io/OAuth-NL-profiel/>

Latest published version:

<https://gitdocumentatie.logius.nl/publicatie/api/oauth/>

Latest editor's draft:

<https://logius-standaarden.github.io/OAuth-NL-profiel/>

Previous version:

<https://gitdocumentatie.logius.nl/publicatie/api/oauth/v1.1.0/>

Editor:

Logius Standaarden ([Logius](#))

Authors:

Jaron Azaria ([Logius](#))

Martin Borgman ([Kadaster](#))

Marc Fleischeuers ([Kennisnet](#))

Alexander Green ([Logius](#))

Peter Haasnoot ([Logius](#))

Heiko Hudig ([Logius](#))

Martin van der Plas ([Logius](#))

Stas Mironov ([Logius](#))

Leon van der Ree ([Logius](#))

Bob te Riele ([RvIG](#))

Remco Schaar ([Logius](#))

Frank Terpstra ([Geonovum](#))

Jan Jaap Zoutendijk ([Rijkswaterstaat](#))

Participate:

[GitHub Logius-standaarden/OAuth-NL-profiel](#)

[All issues](#)

[File an issue](#)

[Commit history](#)

[Pull requests](#)

This document is also available in these non-normative format: [PDF](#)



This document is licensed under

[Creative Commons Attribution 4.0 International Public License](https://creativecommons.org/licenses/by/4.0/)

Status of This Document

This is a draft that could be altered, removed or replaced by other documents. It is not a recommendation approved by TO.

Table of Contents

Status of This Document

Conformance

Requirements Notation and Conventions

Terminology

Conformance

Abstract

Dutch government Assurance profile for OAuth 2.0

Usecases

Introduction

Resource Server

Authorization Server

Client

Use case: Client credentials flow

Step 1. Client Authentication

Step 2. Access Token Response

Step 3. Resource interaction

Use case: Authorization code flow

Step 1. Authorization initiation

Step 2. Authorization Request

Step 3. User Authorization and consent

Step 4. Authorization Grant

Step 5. Access Token Request

Step 6. Access Token Response

Step 7. Resource interaction

Use case: Token exchange

Step 1: Token Exchange Request

Step 2: Access Token Response

Step 3: Resource Interaction

1. Client Profiles

1.1 Client Types

1.1.1 Full Client with User Delegation

1.1.2 Native Client with User Delegation

1.1.3 Direct Access Client

1.2 Connection to the Authorization Server

1.2.1 Requests to the Authorization Endpoint

1.2.2 Response from the Authorization Endpoint

1.2.3 Requests to the Token Endpoint

1.2.4 Client Keys

2. Authorization Server Profile

2.1 Connections with clients

2.1.1 Grant types

2.1.2 Client authentication

2.1.3 Dynamic Registration

2.1.4 Client Approval

2.1.5 Discovery

2.1.6 Revocation

2.1.7 PKCE

2.1.8 Redirect URIs

2.1.9 RefreshTokens

2.1.10 Token Response

2.2 Connections between Authorization Servers and Protected Resources

2.2.1 JSON Web Token (JWT) Bearer Tokens <!-- kan ik dit afkorten naar JWT Bearer Tokens?>

2.2.2 Introspection

2.3 Response to Authorization Requests

2.4 Token Lifetimes

2.5 Scopes

2.5.1 Claims for Authorization Outside of Delegation Scenarios

3. Protected Resource Profile

3.1 Protecting Resources

3.2 Connections with Clients

3.3 Connections with Authorization Servers

4. Advanced OAuth Security Options

- 4.1 Pushed Authorization Requests (PAR)
- 4.2 Proof of Possession Tokens (PoP)
- 4.3 Rich Authorization Requests

5. Security Considerations

A. References

- A.1 Normative references
- A.2 Informative references

§ Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *NOT RECOMMENDED*, *OPTIONAL*, *RECOMMENDED*, *REQUIRED*, *SHALL*, *SHALL NOT*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in [BCP 14](#) [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

This profile is based upon the International Government Assurance Profile (iGov) for OAuth 2.0 [[iGOV.OAuth2](#)] as published by the [OpenID Foundation](#). It should be considered a fork of that profile, as the iGov profile is geared more towards the American situation; in the Netherlands the European Union context applies.

All requirements from [[iGOV.OAuth2](#)] apply to the corresponding NLgov roles (client, authorization server, protected resource) unless explicitly overridden or extended in this document. Readers *MUST* consult [[iGOV.OAuth2](#)] alongside this document.

§ Requirements Notation and Conventions

The key words "*MUST*", "*MUST NOT*", "*REQUIRED*", "*SHALL*", "*SHALL NOT*", "*SHOULD*", "*SHOULD NOT*", "*RECOMMENDED*", "*NOT RECOMMENDED*", "*MAY*", and "*OPTIONAL*" in this document are to be interpreted as described in [[rfc2119](#)] .

All uses of [JSON Web Signature (JWS)] [[rfc7515](#)] and [JSON Web Encryption (JWE)] [[rfc7516](#)] data structures in this specification utilize the JWS Compact Serialization or the JWE Compact Serialization; the JWS JSON Serialization and the JWE JSON Serialization are not used.

§ Terminology

This specification uses the terms "Access Token", "Authorization Code", "Authorization Endpoint", "Authorization Grant", "Authorization Server", "Client", "Client Authentication", "Client Identifier", "Client Secret", "Grant Type", "Protected Resource", "Redirection URI", "Refresh Token", "Resource Owner", "Resource Server", "Response Type", and "Token Endpoint" defined by [OAuth 2.0] [[rfc6749](#)], the terms "Claim Name", "Claim Value", and "JSON Web Token (JWT)" defined by [JSON Web Token (JWT)] [[rfc7519](#)], and the terms defined by [OpenID Connect Core 1.0] [[OpenID.Core](#)].

§ Conformance

Where [[iGOV.OAuth2](#)] refers to "iGov-compliant" components, this profile substitutes "NLgov-compliant". All conformance requirements of [[iGOV.OAuth2](#)] section 1.3 apply accordingly.

This document is an adaptation of the '[International Government Assurance Profile \(iGov\) for OAuth 2.0 - Draft 03](#)' (hereinafter: the [iGov-profile](#)) of the OpenID Foundation. This does not indicate an endorsement by the OpenID Foundation. In as far as the iGov-profile is incorporated in this document, the [OpenID Copyright License](#) applies.

Abstract

The OAuth 2.0 protocol framework defines a mechanism to allow a resource owner to delegate access to a protected resource for a client application.

This specification profiles the OAuth 2.0 protocol framework to increase baseline security, provide greater interoperability, and structure deployments in a manner specifically applicable, but not limited to consumer-to-government deployments in the Netherlands.

Organization / Committee	Version number	Official status	Date
Forum Standaardisatie	-	reported	16-03-2016
Working group	1.0	definitive approved version	15-07-2019

Organization / Committee	Version number	Official status	Date
Forum Standaardisatie	1.0	'comply of explain' standard (mandatory open standard)	09-07-2020
KP-API Working group	1.1.0-rc.1	working version / final draft by 'Working Group'	13-05-2024
OAuth-NL Working group	1.1.0-rc.2	proposed version - updated after public consultation	10-07-2024
MIDO programmeringstafel	1.1.0	definitive approved version	03-12-2024
Forum Standaardisatie	-	-	tbd

§Dutch government Assurance profile for OAuth 2.0

This profile is based upon the [International Government Assurance Profile \(iGov\) for OAuth 2.0](#) as published by the [OpenID Foundation](#). It should be considered a fork of this profile as the iGov profile is geared more towards the American situation and in the Netherlands we have to deal with an European Union context.

We have added the chapter [Use cases](#) to illustrate the specific use case the iGov-NL profile is aimed at. Starting with chapter [Introduction](#) we follow the structure of the iGov profile. Where we do not use content from iGov we use ~~strikethrough~~ to indicate it is not part of iGov-NL.

iGov-NL : Additional content

Content added for the iGov-NL profile is indicated like this.

[The Governance of this standard](#) is described by the API-Standaarden beheermodel in a [separate repository](#) and published by Logius (api@logius.nl).

§Usecases

There are two use cases: The *client credentials* flow and the *authorization code* flow. In two sections below we will elaborate on these, first we will introduce some common concepts.

§ Introduction

For the Client credentials flow and Authorization code flow usecases to work properly the following application building blocks need to be in place:

1. the **Resource Server** (usually described as the API)
2. the **Authorization Server**
3. the **Client** (application)

§ Resource Server

The service is provided by a public/governmental organization. Assumed is the Resource Server is known (by the Authorization Server) prior to actual authorization of the User. A Resource Server is assumed to possess a means for identification of the Resource Server and/or encrypted information, optionally by using a PKI certificate. Furthermore, a Resource Server is assumed to be provided over HTTP using TLS, other protocols are out of scope for this profile.

§ Authorization Server

An Authorization Server is available, operated by either an independent trusted third-party or the service provider itself. Only a single Authorization Server is in use. The Authorization Server is trusted by the Resource Server. The Authorization Server can identify and authorize the User. In case the User has no direct relationship to the Authorization Server, it can forward the User to an IDP trusted by both the Authorization Server as well as the User. Alternatively, the Authorization Server can otherwise identify and authorize the User and is trusted by that User.

§ Client

The User uses a client, which can be any arbitrary application decided upon by the User. Assumed is that the User trusts this client for interaction with the service. The authorization server has at least low trust in the client when the client is either public or semi-confidential. Assumptions is that the Client is aware of the specifications of the API and authorization is required. The Client is either using a user-agent, typically a browser, or the relevant parts are integrated into the Client application.

Note: Web-applications by default use the system-browser on a User's device as user-agent. Typically a native application ("*mobile app*") either starts a system browser as user-agent or uses an *in-app* browser. See RFC 8252 for more information on implementation of native applications. Clients can also be 'machine clients' types.

§ Use case: Client credentials flow

The client credentials flow can be used in usecases where there is an Client to Resource server connection where no user information is needed by the resource server. Two examples are:

- An application does a system API call. For instance a ping service to see if an API is available. The user does not need to be logged in for this and there is no relation to the identity of the end user.
- A batch application processes a large number of transactions asynchronously at a later scheduled time. The original access_tokens of the preceding synchronous process is no longer available. The flow for such a machine to machine interaction is shown in the figure below.

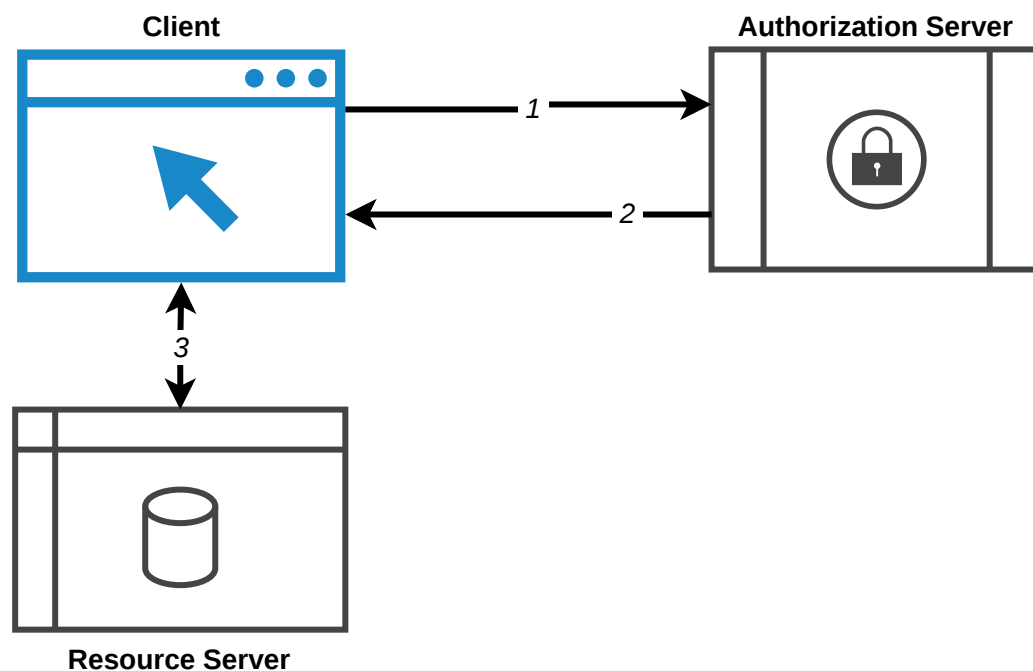


Figure 1 Use case Client credentials flow

§ Step 1. Client Authentication

Using the client credentials, the client sends a Authentication Request to the Authorization Server's token Endpoint. It does so using the Client authentication as pre-registered. The Authorization Server receives and validates the Authentication Request.

§ Step 2. Access Token Response

The Authorization Server authenticates the client and if valid responds to the client with an Access Token Response. The Authorization server issues an Access Token, specific to the requested authorization. The client receives the Access Token and can use the Access Token to send requests to the Service API.

§ Step 3. Resource interaction

The Client can now send (a) request(s) to the Service, on behalf of itself. It does so by sending requests to the Resource Server, along with the Access Token. The Resource Server uses the Access Token for its access control decision. The Resource Server responds based on these decisions to the Client. The contents and protocol of the Resource Request and Resource Response are out of scope of this profile.

Direct access clients that are using the client credentials grant type and are not using OpenIDConnect are also allowed to use an X.509 certificate to authenticate with the authorization server's token endpoint. This flow is compatible with OAuth 2.0 due to section 2.3.2 of [\[rfc6749\]](#).

§ Use case: Authorization code flow

In this use case a (public/governmental) service is offered via an API. The service will be consumed by the User using a client, that can be any arbitrary, non-trusted application. For provisioning the service, the service provider requires an identifier of the User. The identifier of the User can be either an arbitrary (self-registered) identifier or a formal identifier (citizen number or

other restricted, registered ID). Upon service provisioning, the service uses the identifier of the User for access control within the service.

A Client wishes to send a request to an API, on behalf of the User. The API requires to have a trusted identification and *authorization* of the User, before providing the Service. A Client has pre-registered with the Authorization Endpoint and has been assigned a client_id.

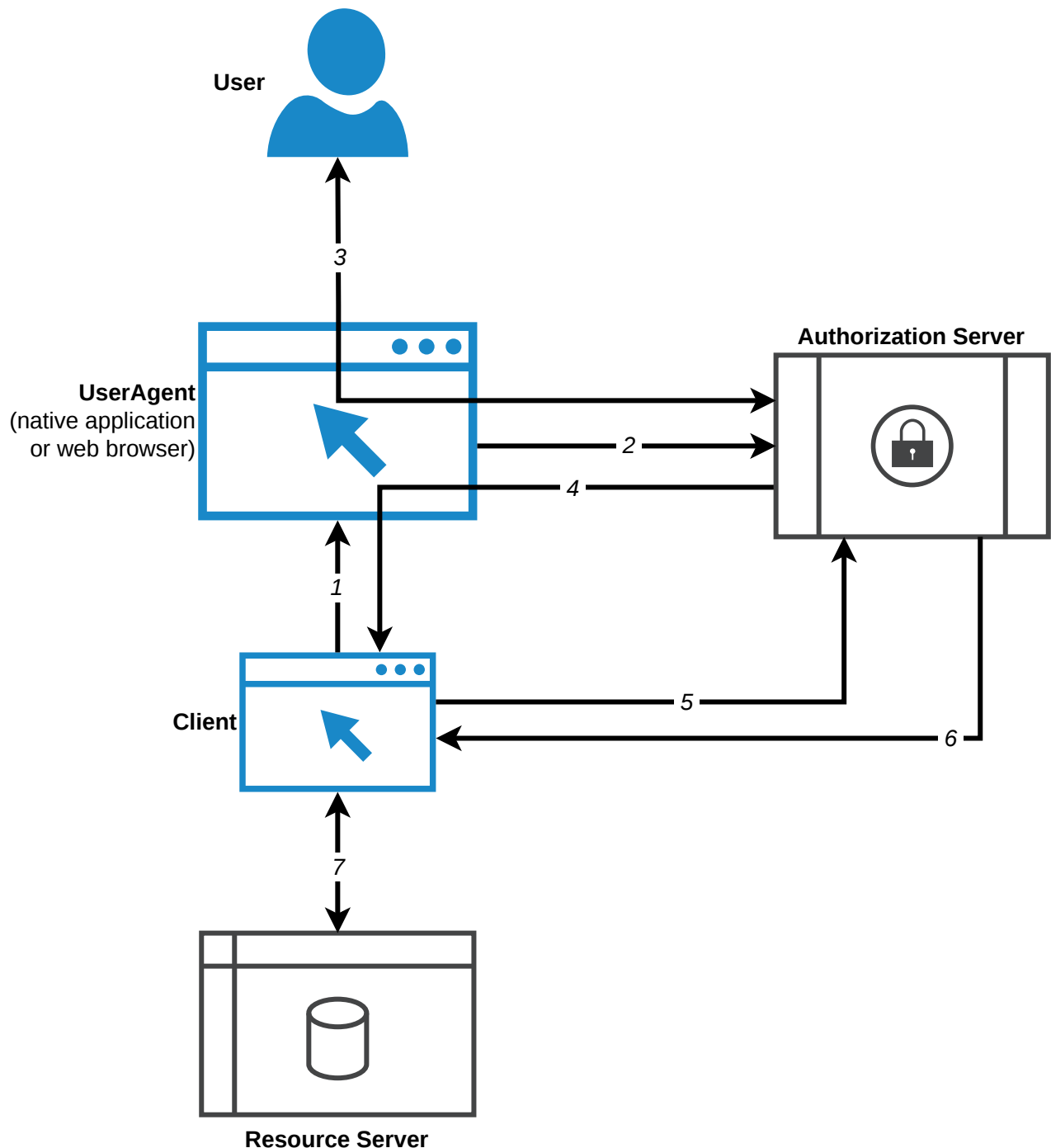


Figure 2 Use case Authorization code flow

The normal flow, that is without any error handling, is described below.

§ Step 1. Authorization initiation

As the client does not yet have a (valid) access token for this Service, it's first step is to obtain one. Therefore it sends an Authorization Request to the Authorization Server's Authorization Endpoint. It does so by redirecting / initiating the user-agent with the Authorization Request to the Authorization Endpoint. The Authorization request holds further details, as specified in this profile.

NOTE: Extra security consideration

When the Authorization Server supports [OAuth 2.0 Pushed Authorization Requests](#) (PAR), the client may first use PAR (or is required to use it, see `require_pushed_authorization_requests` in [Authorization Server Metadata](#)). The client can initiate the flow by pushing a POST request with the parameters to the `pushed_authorization_request_endpoint`. The Authorization Server responds to the client with a `request_uri` containing a reference. The client will then use this `request_uri` as the redirect.

§ Step 2. Authorization Request

The user-agent sends the Authorization request to the Authorization Endpoint. The Authorization Server receives and validates the request.

§ Step 3. User Authorization and consent

The Authorization Server identifies the Resource Owner (often, but not necessarily, the User) and obtains authorization and consent from the Resource Owner for using the client to access the Service. The method and means for identification, as well as how to obtain authorization and consent from the Resource Owner for the request, are implementation specific and explicitly left out of scope of this profile. Note that if the User and Resource Owner are one and the same, the Authorization Server will have to authenticate the User in order to reliably identify the User as Resource Owner before obtaining the authorization and consent.

§ Step 4. Authorization Grant

Note: applicable to the Authorization Code Flow only. The Authorization Server redirects the user-agent back to the Client, with a Authorization Response. This Authorization Response holds an Authorization Grant and is send to the `redirect_uri` endpoint from the Authorization request.

§ Step 5. Access Token Request

The Client receives the Authorization Response from the user-agent. Using the Authorization Grant from the response, the client sends a Token Request to the Authorization Server's token Endpoint. It does so using the Client authentication as pre-registered. The Authorization Server receives and validates the Token Request.

§ Step 6. Access Token Response

The Authorization Server responds to the client with an Access Token Response. This response contains an Access Token, specific to the requested authorization. The client receives and validates the Access Token and can use the Access Token to send requests to the Service API.

§ Step 7. Resource interaction

The Client can now send (a) request(s) to the Service, on behalf of its User. It does so by sending requests to the Resource Server, along with the Access Token. The Resource Server uses the Access Token for its access control decision and any customization of the service or data for the User, if applicable. The Resource Server responds based on these decisions to the Client. The Client can inform and interact with the User based on the information received from the Resource Server. The contents and protocol of the Resource Request and Resource Response are out of scope of this profile.

§ Use case: Token exchange

Token exchange is useful when the resource server requires a different token than the one(s) the client originally received during a prior interaction. To obtain this different token, the client can use token exchange [rfc8693]. It can be used for both impersonation and delegation, as specified in that RFC.

The flow for such a machine to machine interaction is shown in the figure below.

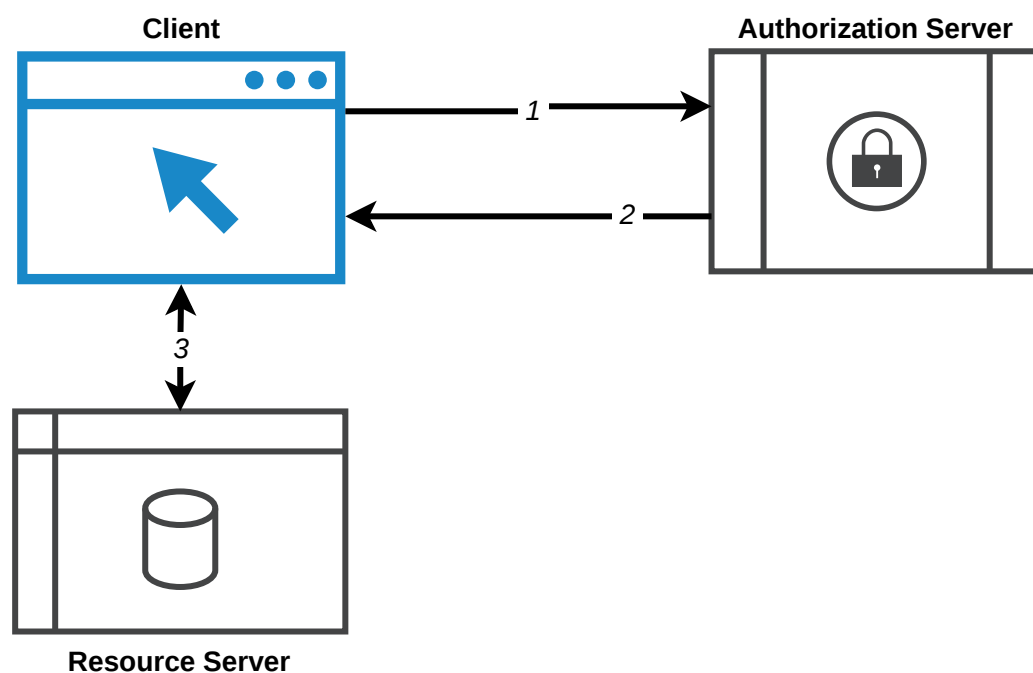


Figure 3 Use case token exchange

Note that the method by which the client received the original token(s)—either directly from an OAuth or SAML token server, or indirectly via other client applications—is not part of the token exchange process itself and, therefore, is not depicted in the diagram.

§ Step 1: Token Exchange Request

Using client credentials, the client sends a token exchange request to the Authorization Server's token endpoint. At a minimum, this request includes the subject token and the token exchange grant type. Optionally, the request may also include an actor token, audience, resource, and scopes.

§ Step 2: Access Token Response

The Authorization Server authenticates the client and validates the tokens and requested access, according to established policies. If valid, it issues a new token (typically an access token). This new token contains an "act" or "may_act" claim, linking the new token to the original subject token(s) as outlined in RFC 8693, as well as other claims relevant to the token type (e.g., an access token). The response may also include a refresh token, expiration details, and scopes. The client receives the new (access) token and can use it to make requests to the service API.

§ Step 3: Resource Interaction

The client can now use the new token to send requests to the service. These requests are directed to the Resource Server, which evaluates the access token as part of its access control decisions (e.g., by using the subject in the access token). The Resource Server responds based on these access control decisions. The specifics of the Resource Request and Resource Response are beyond the scope of this document.

§1. Client Profiles

§1.1 Client Types

§ 1.1.1 Full Client with User Delegation



In addition to `private_key_jwt`, the client authentication method `tls_client_auth` [rfc8705] MAY also be used.

§ 1.1.2 Native Client with User Delegation

§ 1.1.3 Direct Access Client

§ 1.2 Connection to the Authorization Server

§ 1.2.1 Requests to the Authorization Endpoint

§ 1.2.2 Response from the Authorization Endpoint

§ 1.2.3 Requests to the Token Endpoint

§ 1.2.4 Client Keys

Public clients *MUST* apply PKCE, as per [[rfc7636](#)]. As `code_challenge` the S256 method *MUST* be applied. Effectively this means that browser-based and native clients *MUST* include a cryptographic random `code_verifier` of at least 128 bits of entropy and the `code_challenge_method` with the value S256.

Request fields:

`client_id` : Mandatory. *MUST* have the value as obtained during registration.

`scope` : Optional.

`response_type` : Mandatory. *MUST* have value `code` for the Authorization Code Flow.

`redirect_uri` : Mandatory. *MUST* be an absolute HTTPS URL, pre-registered with the Authorization Server.

`state` : Mandatory. Do not use the SessionID secure cookie for this.

`code_challenge` : Mandatory when using a native app or browser-based app as user-agent.

`code_challenge_method` : Mandatory when `code_challenge` is used. *MUST* use the value S256.

When the Authorization Server supports OAuth 2.0 Pushed Authorization Requests (PAR) [rfc9126], the client *MAY* use PAR (or *MUST*, if `require_pushed_authorization_requests` is set in the Authorization Server Metadata). The client pushes the authorization parameters to the `pushed_authorization_request_endpoint` and uses the returned `request_uri` as the redirect.

Response parameters:

`code` : Mandatory. *MUST* be a cryptographic random value, using an unpredictable value with at least 128 bits of entropy.

`state` : Mandatory. *MUST* be a verbatim copy of the value of the `state` parameter in the Authorization Request.

When using the JWT assertion for client authentication, the assertion *MUST* use the claims as specified in [rfc7521] and [rfc7523].

In addition to `private_key_jwt`, the client authentication method `tls_client_auth` [rfc8705] *MAY* also be used. Private Key JWT is a method of client authentication where the client creates and signs a JWT using its own private key, as described in [rfc7521] and [rfc7523] and referenced by OpenID Connect and FAPI 2.0 Security Profile.

The Authorization Server *SHOULD* support the PS256 signing algorithm [rfc7518] for the signing of the `private_key_jwt`.

Where the Authorization Server, Resource Server, and client are not operated under the responsibility of the same organisation, each party *MUST* use PKIoverheid certificates with OIN.

The PKIoverheid certificate *MUST* be included either as an `x5c` or as an `x5u` parameter, as per [rfc7517] sections 4.6 and 4.7. Parties *SHOULD* at least support the inclusion of the certificate as `x5c` for maximum interoperability. Parties *MAY* agree to use `x5u`, for instance for communication within specific environments.

§2. Authorization Server Profile

All servers *MUST* conform to applicable recommendations found in the Security Considerations sections of [rfc6749] and those found in the "OAuth Threat Model Document" [rfc6819] .

The authorization server *MUST* protect all communications to and from its OAuth endpoints using TLS.

§2.1 Connections with clients

§ 2.1.1 Grant types

The authorization server *MUST* support the `authorization_code` , and *MAY* support the `client_credentials` grant types as described in [Section 2](#). The authorization server *MUST* limit each registered client (identified by a client ID) to a single grant type only, since a single piece of software will be functioning at runtime in only one of the modes described in [Section 2](#). Clients that have multiple modes of operation *MUST* have a separate client ID for each mode.

iGov-NL : Additional content

Token exchange grant type [rfc8693] *SHOULD* be supported by the authorization server. This is used to translate a third party token (OAuth or SAML). For example, exchanging a DigiD or eHerkenning SAML token for an OAuth token.

When token exchange is not supported, SAML bearer grant [rfc7522] *MAY* be used as alternative.

§ 2.1.2 Client authentication

§ 2.1.3 Dynamic Registration

§ 2.1.4 Client Approval

§ 2.1.5 Discovery

§ 2.1.6 Revocation

§ 2.1.7 PKCE

§ 2.1.8 Redirect URIs

§ 2.1.9 RefreshTokens

§ 2.1.10 Token Response

§2.2 Connections between Authorization Servers and Protected Resources

§ 2.2.1 JSON Web Token (JWT) Bearer Tokens <!-- kan ik dit afkorten naar JWT Bearer Tokens?>

§ 2.2.2 Introspection

§2.3 Response to Authorization Requests

§2.4 Token Lifetimes

§2.5 Scopes

§ 2.5.1 Claims for Authorization Outside of Delegation Scenarios

In addition to `private_key_jwt`, the client authentication method `tls_client_auth` [[rfc8705](#)] *MAY* also be used.

Authorization servers ***MUST*** support dynamic client registration.

In this version of the NLgov profile, Authorization Servers *SHOULD* support dynamic client registration. Depending on how the future authentication architecture of the Dutch government develops in regards to OAuth, this may be revisited in a future revision.

The current requirement fits an architecture where there is a limited number of widely used authorization servers. If in practice a very large number of authorization servers with limited use emerges, mandatory support for dynamic client registration is considered a large burden for such servers.

The `tls_client_auth` method *SHOULD* be listed in the Authorization Server discovery metadata where supported.

Refresh tokens issued to public clients *MUST* be either sender-constrained or one-time use, per the OAuth 2.1 Authorization Framework [[draft-ietf-oauth-v2-1](#)] and [[rfc9700](#)] section 2.2.2.

The Token Response has the following contents:

`access_token` : Mandatory. Structured access token (JWT Bearer token). The JWT *MUST* be signed.

`token_type` : Mandatory. For a JWT Bearer token the value is Bearer, as per [[rfc6750](#)].

`refresh_token` : Under this profile, refresh tokens are supported.

`expires_in` : Optional. Lifetime of the access token, in seconds.

`scope` : Optional. Scope(s) of the granted access token; multiple scopes are separated by whitespace. *MAY* be omitted if identical to the scope requested.

For best practices on token lifetime see section [Token Lifetimes](#).

The `sub` claim *MUST* be present in access tokens issued under this profile.

The Authorization Server *SHOULD* support the PS256 signing algorithm [[rfc7518](#)] for signing JWT Bearer Tokens.

Where the Authorization Server, Resource Server, and client are not operated under the responsibility of the same organisation, each party *MUST* use PKIoverheid certificates with OIN for encryption.

How to select or obtain the key for encryption of an access token is out of scope of this profile. The resource parameter from Resource Indicators for OAuth 2.0 [[rfc8707](#)] *MAY* be used to indicate the applicable resource server.

If there is a need to include resource owner memberships in roles and groups that are relevant to the resource being accessed, or entitlements assigned to the resource owner for the targeted resource that the authorization server knows about, the authorization server *SHOULD* include such attributes as claims in a JWT access token as defined in section 2.2.3.1 of [[rfc9068](#)].

§3. Protected Resource Profile

§3.1 Protecting Resources

§3.2 Connections with Clients

§3.3 Connections with Authorization Servers

A protected resource ***MAY*** accept bearer tokens passed using the query parameter method.

A Protected Resource under this profile *MUST NOT* accept access tokens passed using the query parameter method.

A Protected Resource under this profile *SHOULD* verify if the client is the Authorized party (azp) when client authentication is used. See section [Advanced OAuth Security Options](#) as well.

Where the optional token attributes (aud, sub, azp, etc.) are already present within the access token, token introspection is not required for those attributes. For further details see the [[NLgov.OpenID](#)] specification.

§4. Advanced OAuth Security Options

The preceding portions of this OAuth profile provide a level of security adequate for a wide range of use cases, while still maintaining relative ease of implementation and usability for developers, system administrators, and end users. The following are some additional security measures that can be employed for use cases where elevated risks justify the use of additional controls at the expense of implementation effort and usability. This section also addresses future security capabilities, currently in the early draft stages, being added to the OAuth standard suite.

§4.1 Pushed Authorization Requests (PAR)



Traditionally, OAuth 2.0 authorization requests are sent via front-channel communication (e.g., browser redirects), which exposes sensitive parameters to potential tampering or interception. PAR [rfc9126] addresses these vulnerabilities by allowing clients to push authorization requests directly to the authorization server over a secure back-channel. [FAPI 2.0 Security Profile](#) also includes this feature as of version 2.0. Below are some of the issues it alleviates:

- **Lack of Integrity and Authenticity:** Authorization request parameters sent as URI query parameters are vulnerable to tampering. Attackers can modify values like scope or redirect_uri, potentially altering the context of transactions or access permissions. Such sensitive data in front-channel requests can be intercepted or phished, compromising client credentials or authorization codes. Attackers can exploit the request_uri parameter by injecting malicious URIs, leading to unauthorized access or token leakage.
- **Lack of Confidentiality:** Although HTTPS protects the authorization endpoint, request parameters pass through the user agent in the clear, risking exposure via browser logs, referrer headers, or other leaks. This is particularly problematic for sensitive data like personally identifiable information (PII).
- **Size Limitations:** Large authorization requests with fine-grained permissions can exceed URL size limits, causing processing errors.
- **Delayed Client Authentication:** Traditional flows delay client authentication until after user interaction, making it harder to detect and reject illegitimate requests early.

To combat this PAR allows clients to push authorization requests directly to the authorization server over a secure back-channel (HTTPS), effectively preventing tampering. For higher security, PAR can be combined with JWT-based Request Objects for cryptographic signing and optional encryption. Furthermore, by moving sensitive data from the front-channel (user agent) to the back-channel, PAR ensures that request parameters are not exposed to the browser or

third parties, mitigating leakage risks. Also, PAR enables the authorization server to authenticate the client before any user interaction, allowing early detection and rejection of illegitimate requests, such as spoofing or tampering attempts.

§4.2 Proof of Possession Tokens (PoP)



Proof of possession can be implemented using various methods. One example is mutual TLS, where the client authenticates using a PKIoverheid certificate. The authorized party (azp) can then be verified against the client certificate.

As an alternative, the authorization server *MAY* include a `cnf` parameter in the JWT access token, as per [rfc7800]. The key referenced in `cnf` can be validated using client authentication such as `private_key_jwt` or `tls_client_auth` [rfc8705].

§4.3 Rich Authorization Requests



[OAuth 2.0 Rich Authorization Requests](#) is an extension that provides a way for clients to request and obtain fine-grained authorization from resource owners such as end users during the Authorization Code Flow. In traditional OAuth flows, clients typically request access to a set of scopes from a Resource Server. The Resource Owner then grants access to the resources to the client. However, this approach allows limited granular control over the access granted to a client and can lead to over-provisioning of access, which poses various security risks. With RAR, clients can pass an `authorization_details` claim with additional details that allow for more fine-grained authorization. This also allows for the Resource Server to implement fine-grained authorization for specific requests. Think of one-time payment approvals, document signing or requesting or approving (access to a) specific cases or documents.

According to the RFC, `authorization_details` requires just one field, `type`, which determines the allowable contents of the `authorization_details`. The value is unique for the described API in the context of the Authorization Server. The underlying RFC defines a set of common data fields that are designed to be usable across different types of APIs. Fields like `locations`, `datatypes`, `identifier` and `privileges` can be added in the `authorization_details` parameter.

As mentioned in [RFC9396]:

In case of authorization requests as defined in [RFC6749], implementers *MAY* consider using pushed authorization requests [RFC9126] to improve the security, privacy, and reliability of the flow.

In case more data, or more recent data, is required for fine-grained authorization then one *MAY* include the OpenID AuthZEN standard in the `authorization_details`.

§5. Security Considerations



Response parameters:

`code` : Mandatory. *MUST* be a cryptographic random value, using an unpredictable value with at least 128 bits of entropy.

`state` : Mandatory. *MUST* be a verbatim copy of the value of the `state` parameter in the Authorization Request.

§A. References

§A.1 Normative references

[draft-ietf-oauth-v2-1]

The OAuth 2.1 Authorization Framework. Dick Hardt; Aaron Parecki; Torsten Lodderstedt. IETF. URL: <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1>

[NLgov.OpenID]

NLgov Assurance profile for OpenID Connect 1.0. Remco Schaar, Frank van Es, Joris Joosten, Jan Geert Koops. Logius. September 18, 2023. URL: <https://gitdocumentatie.logius.nl/publicatie/api/oidc/>

[OpenID.FAPI2.0]

FAPI 2.0 Security Profile. D. Fett, D. Tonge, J. Heenan. OpenID foundation. February 22, 2025. URL: https://openid.net/specs/fapi-security-profile-2_0-final.html

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc2119/>

[rfc6749]

[The OAuth 2.0 Authorization Framework](#). D. Hardt, Ed. IETF. October 2012. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc6749/>

[rfc6750]

[The OAuth 2.0 Authorization Framework: Bearer Token Usage](#). M. Jones; D. Hardt. IETF. October 2012. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc6750/>

[rfc6819]

[OAuth 2.0 Threat Model and Security Considerations](#). T. Lodderstedt, Ed.; M. McGloin; P. Hunt. IETF. January 2013. Informational. URL: <https://www.rfc-editor.org/info/rfc6819/>

[rfc7517]

[JSON Web Key \(JWK\)](#). M. Jones. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7517/>

[rfc7518]

[JSON Web Algorithms \(JWA\)](#). M. Jones. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7518/>

[rfc7521]

[Assertion Framework for OAuth 2.0 Client Authentication and Authorization Grants](#). B. Campbell; C. Mortimore; M. Jones; Y. Goland. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7521/>

[rfc7522]

[Security Assertion Markup Language \(SAML\) 2.0 Profile for OAuth 2.0 Client Authentication and Authorization Grants](#). B. Campbell; C. Mortimore; M. Jones. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7522/>

[rfc7523]

[JSON Web Token \(JWT\) Profile for OAuth 2.0 Client Authentication and Authorization Grants](#). M. Jones; B. Campbell; C. Mortimore. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7523/>

[rfc7636]

[Proof Key for Code Exchange by OAuth Public Clients](#). N. Sakimura, Ed.; J. Bradley; N. Agarwal. IETF. September 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7636/>

[rfc7800]

[Proof-of-Possession Key Semantics for JSON Web Tokens \(JWTs\)](#). M. Jones; J. Bradley; H. Tschofenig. IETF. April 2016. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7800/>

[RFC8174]

[Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words](#). B. Leiba. IETF. May 2017. Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc8174/>

[rfc8693]

[OAuth 2.0 Token Exchange](#). M. Jones; A. Nadalin; B. Campbell, Ed.; J. Bradley; C. Mortimore. IETF. January 2020. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc8693/>

[rfc8705]

[OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens](#). B. Campbell; J. Bradley; N. Sakimura; T. Lodderstedt. IETF. February 2020. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc8705/>

[rfc8707]

[Resource Indicators for OAuth 2.0](#). B. Campbell; J. Bradley; H. Tschofenig. IETF. February 2020. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc8707/>

[rfc9068]

[JSON Web Token \(JWT\) Profile for OAuth 2.0 Access Tokens](#). V. Bertocci. IETF. October 2021. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc9068/>

[rfc9126]

[OAuth 2.0 Pushed Authorization Requests](#). T. Lodderstedt; B. Campbell; N. Sakimura; D. Tonge; F. Skokan. IETF. September 2021. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc9126/>

[RFC9396]

[OAuth 2.0 Rich Authorization Requests](#). T. Lodderstedt; J. Richer; B. Campbell. IETF. May 2023. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc9396/>

[rfc9700]

[Best Current Practice for OAuth 2.0 Security](#). T. Lodderstedt; J. Bradley; A. Labunets; D. Fett. IETF. January 2025. Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc9700/>

§A.2 Informative references

[iGOV.OAuth2]

[International Government Assurance Profile \(iGov\) for OAuth 2.0](#). J. Richer, M. Varley, P. Grassi. OpenID foundation. October 5, 2018. URL: https://openid.net/specs/openid-igov-oauth2-1_0-03.html

[OpenID.Core]

[OpenID Connect Core 1.0](#). N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, C. Mortimore. OpenID foundation. November 8, 2014. URL: https://openid.net/specs/openid-connect-core-1_0.html

[rfc7515]

[JSON Web Signature \(JWS\)](#). M. Jones; J. Bradley; N. Sakimura. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7515/>

[rfc7516]

[JSON Web Encryption \(JWE\)](#). M. Jones; J. Hildebrand. IETF. May 2015. Proposed Standard.

URL: <https://www.rfc-editor.org/info/rfc7516/>

[rfc7519]

[JSON Web Token \(JWT\)](#). M. Jones; J. Bradley; N. Sakimura. IETF. May 2015. Proposed

Standard. URL: <https://www.rfc-editor.org/info/rfc7519/>